Laboratory Exercise 2
Logic Optimization, Universal Gates and Parity

Introduction

This laboratory exercise accompanies Lecture 3 (Logic Optimization using Karnaugh Maps). You will minimize a logic function with a Karnaugh map, implement it with NAND gates only (a two-level NAND-NAND circuit), take advantage of "don't-care" conditions, and finally design a parity generator and checker with exclusive-OR gates in Quartus. As in Labs 0 and 1, no VHDL is needed. The session is two hours long, so the paper work is done in the pre-lab.

Objectives

1. To simplify a four-variable Boolean function using a Karnaugh map.
2. To implement a Sum-of-Products function using only NAND gates and to verify it on the breadboard.
3. To see how don't-care conditions reduce the size of a circuit.
4. To design a parity generator and a parity checker and to detect a single-bit error.

Equipment Required

- DC Power Supply (+5V), Digital Multimeter
- Breadboard and Jumper Wires
- TTL ICs: 74LS00 (Quad 2-Input NAND) ×2, 74LS04 (Hex Inverter)
- Resistors: 330Ω (for LEDs and pull-down)
- DIP switches, LEDs (Red/Green)
- Intel DE0-CV board and a PC with Quartus Prime
- (Optional) Logisim or Logisim-evolution: a light-weight tool for drawing and simulating small circuits at home or in the laboratory. It cannot program the DE0-CV board, so Quartus Prime is still needed for the board.

Pre-lab (complete before the session)

Complete Part I(a)–(d), Part II(a)–(b) and Part III(a) *before* the laboratory session. The pin numbers of the 74LS00 and 74LS04 are the same as in Lab 1 (Table 1 of that sheet). You can check your K-maps, expressions and NAND-NAND drawing in **Logisim** (Combinational Analysis window for the minimized SOP of a truth table). Remember to use the **LED indicator stage** (inverter + LED + 330 Ω) for each output that you observe on the breadboard.

Part I: Two-Level NAND-NAND Implementation (about 45 min)

Consider the function

$$F(A, B, C, D) = \sum m(0, 1, 2, 5, 8, 9, 10, 13).$$

Simplify with a K-map

(a) Fill the K-map of Figure 1 with the values of F.

(b) Circle the groups of 1s so that each group is as large as possible and the *minimum number of groups* covers all 1s. Is there any prime implicant that is not needed? _____

(c) Write the minimized SOP expression: $F =$ _____

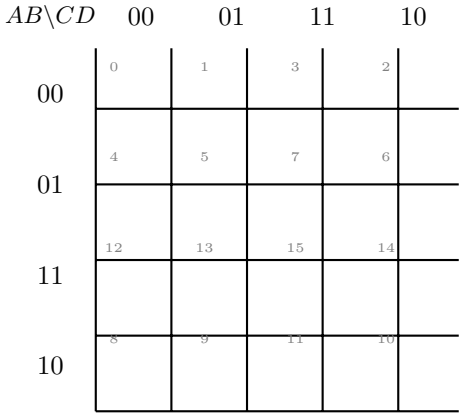


Figure 1: K-map for $F(A, B, C, D)$ (the small numbers are the minterm indices; write 0 or 1 in each cell).

Convert to NAND-NAND form

An SOP expression becomes a NAND-NAND circuit when every AND and the final OR are replaced with NAND gates (Lecture 3, De Morgan's theorem):

$$F = P_1 + P_2 = \big((P_1)' \cdot (P_2)'\big)'$$

An inverter can be made from a NAND gate by connecting its two inputs together.

(d) **(Pre-lab)** Draw the NAND-NAND circuit of your expression. Complemented inputs (B', C', D') must also be built from NAND gates. How many 2-input NAND gates and how many 74LS00 ICs do you need? _____

(e) Build the circuit on the breadboard, with A, B, C, D taken from four switches and F displayed by an LED indicator stage.

(f) Test all 16 input combinations and complete Table 1.

m	A	B	C	D	Exp.	Meas.	m	A	B	C	D	Exp.	Meas.
0	0	0	0	0	1		8	1	0	0	0	1	
1	0	0	0	1	1		9	1	0	0	1	1	
2	0	0	1	0	1		10	1	0	1	0	1	
3	0	0	1	1	0		11	1	0	1	1	0	
4	0	1	0	0	0		12	1	1	0	0	0	
5	0	1	0	1	1		13	1	1	0	1	1	
6	0	1	1	0	0		14	1	1	1	0	0	
7	0	1	1	1	0		15	1	1	1	1	0	

Table 1: Test of the NAND-NAND circuit (Exp. = expected value of F, Meas. = LED reading).

Part II: Don't-Care Conditions (about 30 min)

A BCD digit uses the 4-bit codes 0000 to 1001 (0 to 9). The codes 1010 to 1111 never occur in a BCD system, so the output of the circuit for those inputs is a **don't care**. A BCD "prime-number detector" has an output P that is 1 when the BCD digit is a prime number (2, 3, 5 or 7).

(a) Fill in the K-map of Figure 2 for P, marking the unused codes with an **X**.

(b) Simplify P twice: first *treating all X as 0*, then *using the X's wherever they make a group larger.*

$P_{\text{no DC}} =$ _____

$P_{\text{DC}} =$ _____

(c) Create a new Quartus project `lab2`. Draw both circuits in the Block Editor with the same inputs A, B, C, D (BCD digit, A is the MSB). (The comparison of parts (c)–(e) can also be done in Logisim; the board test of (f) needs Quartus.)

(d) Simulate with a waveform file that counts from 0000 to 1111. Compare the outputs. For which input codes do they differ? Does this matter in a BCD system?

(e) Compile the project. Which circuit is smaller (number of gates)? _____

(f) Implement P_{DC} on the DE0-CV board with SW3–SW0 as A–D and LEDR0 as P. Step through the digits 0 to 9 with the switches and verify that P is lit for $2, 3, 5, 7$ only.

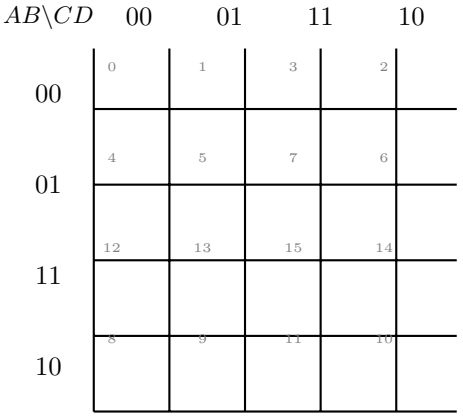


Figure 2: K-map for the BCD prime-number detector P.

Part III: Parity Generator and Checker (about 25 min)

When data are transmitted or stored, a single bit may be flipped by noise. An **even-parity** bit P is added to the data so that the total number of 1s in (data + P) is even. The receiver recomputes the parity; if the result is 1, an error is detected. Figure 3 shows a system for 3 data bits $D_2D_1D_0$.

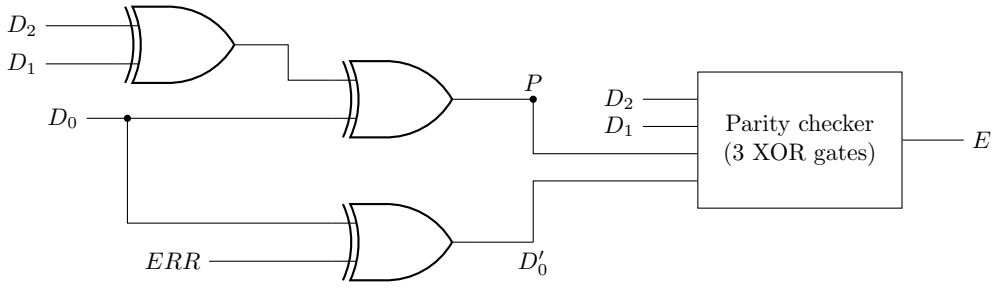


Figure 3: Parity system: generator, error-injection XOR ($ERR = 1$ flips D_0) and checker.

(a) **(Pre-lab)** Write the Boolean expression of the even-parity bit: $P =$ _____
and of the error flag E (using D_2, D_1, D_0', P): $E =$ _____

(b) In the Quartus project (or in Logisim), draw the whole system of Figure 3 as a schematic with XOR gates (6 gates), inputs D_2, D_1, D_0, ERR and outputs P and E.

(c) Simulate: apply the 8 data combinations first with $ERR = 0$ and then with $ERR = 1$, and complete Table 2.

(d) Assign D_2, D_1, D_0, ERR to SW3–SW0, P to LEDR1 and E to LEDR0 and download the circuit to the board. Check a few rows of Table 2 on the board.

(e) What kind of errors can this system detect? What kind can it *not* detect?

D_2	D_1	D_0	P expected	P (sim.)	E ($ERR = 0$)	E ($ERR = 1$)
0	0	0	0			
0	0	1	1			
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

Table 2: Parity generator and checker results.

Extension (take-home, or if time permits)

1. Compare the cost of your minimized circuit of Part I with a direct implementation of the canonical SOP (eight 4-input AND terms and one 8-input OR). How many gate inputs have you saved?
2. Obtain a Product-of-Sums expression for F by grouping the 0s of the K-map and show that it is equal to your SOP expression.
3. Build the parity generator and checker of Part III on the breadboard with two 74LS86 ICs (6 XOR gates) and test it with Table 2.

Check-off

Show your instructor or teaching assistant:

1. Your K-map, minimized expression and NAND-NAND circuit working on the breadboard (Table 1).
2. The simulation of the prime-number detector with and without don't cares.
3. The parity system simulation and the circuit on the board (Table 2).

Updated By: R. Sutthaweekul
Version: 1.0
Release Date: 2026-11-11